Architecting an extensible framework for Gamifying Software Engineering concepts

Sai Krishna Sripada [*]
IIIT-Hyderabad
Hyderabad, India

Y. Raghu Reddy [†]
IIIT-Hyderabad
Hyderabad, India

Shivam Khandelwal [‡]
IIIT-Hyderabad
Hyderabad, India

ABSTRACT

Software engineering activities like code reviews, change management, knowledge management, issue tracking, etc. tend to be heavily process oriented. Gamification of such activities by composing the core activities with game design elements like badges and points can increase developers' interest in performing such activities. While there are various frameworks/applications that assist in gamification, extending the frameworks to add any/all desired game design elements has not been adequately addressed. In this paper, we propose an extensible architectural framework for gamification of software engineering activities where in the game design elements are modeled as services. We create an example instance of our framework by building a prototype for code review activity and note the challenges of designing such an extensible architectural framework. The example instance uses python's Flask micro framework and has five game design elements implemented as services, and exposed using restful APIs.

Keywords

Architecture, Code review, Gamification, Services, Game Design Elements, REST API, Web Hooks

1. INTRODUCTION

In Software Engineering (SE), processes for performing activities like bug hunting, code reviews and inspections, requirement elicitation, change management, knowledge management, issue tracking, etc take huge amount of time and efforts of a developer. Most often such activities require the developers to repeat the same set of tasks over a period of time and may lead to boredom or loss of interest in the activity. In turn this can affect the productivity of the developer and the progress of the project itself. Gamification [10] [PJ15] and Serious games [13] have been suggested as a solution to *persuade* humans to perform the same task again and again. Gamification is the use of *game design elements* and *game thinking* in non-gaming contexts to improve the user's engagement, motivation, and performance. The use of gamification in domains like Education [12], Enterprise healthcare [7], marketing [14], etc. is increasing over the past few years [10]. For example, a case study from Deloitte [1] revealed 37% increase in the users returning to Deloitte Leadership Academy training program every week as a result of gamification of their training activity.

Current research [1][2][7] in gamification is primarily concerned with: (1) improving and building domain specific game design elements, (2) building platforms with a subset of game design elements to support gamification, and (3) identifying the challenges in gamifying an application. In Software Engineering, process intense activities can be considered as good candidates for gamification. Designing an architectural framework that can be used to realize and implement gamified instances of such activities with various game design elements is a challenge. The base framework used to build tools that implement the game design elements may vary. Current tools like *getbadges* [2] and *gitpoints* [3] that support gamification rely on existing tools like *github* as their base framework. As a result, these tools limit the gamification to only github supported SE activities. For example, Requirements elicitation activity may require all the features of collaborative text editing supported by utilities such as "*wikipages*". Code Inspections and Code Reviews may require the underlying "$diffutility$" support. Tools that support design reviews may require underlying support for visualization of data flow diagrams or other UML design diagrams and simulation of these. Hence building an extensible architectural framework that supports a product line approach, where in the game design elements can vary independent of the underlying framework is a challenge. There is some work on how to design game design elements depending on progression of a game and when to reward a player [19]. However, there is not much work on designing or implementing an architectural framework and realizing the framework to build gamified instances of the applications suitable for various SE activities.

In this paper, we propose an extensible service-based architectural framework for gamifying software engineering ac-

[*]saikrishna.sripada@research.iiit.ac.in
[†]raghu.reddy@iiit.ac.in
[‡]shivam.khandelwal@research.iiit.ac.in

ISEC '16, February 18-20, 2016, Goa, India

DOI: http://dx.doi.org/10.1145/2856636.2856649

[1]https://hbr.org/2013/01/how-deloitte-made-learning-a-g
[2]http://www.getbadges.com
[3]http://www.gitpoints.com

tivities. The framework supports inclusion of game design elements in any of the non-gaming applications by using **RESTful** web services. Each of the game design elements is modeled as a *provider* or *consumer* service, i.e., each game design element is realizable as modules that are used to achieve certain objective or use other services to achieve certain objective. A run-time instance of the architectural framework utilizes the service API exposed using $WebHooks$. We also designed the architecture taking into consideration Web 2.0 compliance standards to fit the requirements for mobile displays. Designing a gamified application involves creation of rules and content of the game through Game design elements. Game design elements are key to building applications with a motivational UI for the gamified application. Our framework supports two such classifications of game design elements: *static* and *dynamic*. Static game design elements do not depend on the current state of the user input where as dynamic game design elements depend on game state.

Our contributions in this paper include :

- A list of game design elements classified as static and dynamic game mechanics after performing Scope, Commonality and Variability (SCV) analysis [3].
- An extensible service based architectural framework that can be realized to build gamified applications for SE activities.
- Each game design element is designed to be a RESTful web service. Realization of proposed Services using a Micro framework in Python (Flask) [4].
- A prototype run-time instance built from our proposed architectural framework as proof of concept.

The architectural framework has to be designed with an appropriately decoupled user interface suitable for gamification of activities on mobile as well as desktops. The framework includes support of current tools required for performing SE activities and is designed to be able to include additional tool support as and when required, making the architecture extensible.

2. BACKGROUND

2.1 Serious Games Vs Gamification

A serious game has a set of rules and actions bound together in a cohesive manner for the purpose of training or educating its intended users (players). Players can score and compete with one another based on the given set of rules. The games can be designed for individuals or for teams, where the latter encourages collaborations and facilitates community learning.

On the other hand, Gamification uses elements from game design, but is not a game. Gamification can encourage and engage users, where users can compete with one another or simply challenge one's own self to improve own performance. Gamification can exist without a game. Also in gamification, there may not always be an element of competition. It depends on the game design elements used where as a game usually implies a winner (and a loser), with players competing between each other.

[4]http://flask.pocoo.org/

Military, Health care, Business process management systems [17] were early adopters of serious games. Building a serious game requires creation of a story and scenes for the game. Although serious games have higher potential on the impact of engaging users, the possibility of losing a game may make the user stop using the application, which is counter to the major motivation of building a game. Also the cost involved in building a serious game is usually high.

Gamification of an application or specific activities costs much lower compared to serious games because it does not require creation of stories or scenes. Additionally original intent of the activity is not impacted. The idea of making a user lose or win is optional while gamifying an application. As a result, the risk of losing users' interest is minimal.

Many researchers have provided evidence in favor of gamification, however a recent study of women engagement on StackOverflow [15] showed gamification can be counterproductive, encourage problematic behaviour or discourage minorities. It is always good to look at the community as experts versus novice resources and try to improve the expertise of novice users/ developers rather than basing the community on age, sex, color, etc. In this way, the goal of gamification should be to improve the overall expertise and pull the target audience to perform the repetitive task without loosing interest. We present one such approach where the skills of participants always start from an initial base value, ideally '0' or 'None' for all the game design elements made available in our architectural framework. This may bring in added advantage to already expert users, but this is also needed for the novice programmers/developers/designers/test engineers to observe the community and learn to do things. These young and inexperienced players will continue to learn and be motivated by obtaining the perks of a gamified application without loosing interest while comparing with the experts only based on the technical terms.

Currently, there are both commercial and open-source platforms that assist in gamifying an application. Platforms like *bigdoor*[5] and *badgeville*[6] provide enterprise support but are available at a certain cost. Additionally, similar to open source platforms they do not provide all the game design elements. To build a generic platform that supports gamifying an application, there are challenges identified as part of earlier work done in enterprise systems [2]. We take the learning from this work while proposing a generic platform suitable to build a platform for gamifying SE activities.

2.2 Existing gamified applications

A systematic study published in Information and Software technology journal [10] provides detailed mapping of software engineering processes and gamified applications. It is common to find gamified versions of code review, bug identification tools, etc. on the web these days. However the setup and usage of these tools are still a concern. Out of the tools available for code review, some tools are proprietary, some tools require additional setup (eg. requirement of an "organizational" account on github for gitpoints), and some other tools that can be deployed but are not production ready yet.

All gamified applications are not likely to have the same game design elements. Hence there should be a customizable set of game design elements to choose from and develop a gamified application. To gamify applications in such a dy-

[5]www.bigdoor.com

[6]https://badgeville.com/

Table 1: Classification of Frameworks available and game design elements

Available Frameworks	Game design elements supported.
Badgeville [bad]	Competition, Feedback, Levels & Missions, Profiles, Ranks & Badges, Activity Feed, Ratings & Reviews, Social Login
Bigdoor [big]	Economy/Marketplace, Feedback, Profiles, Progress Bar, Ranks & Badges, Activity Feed, Community Forums, Likes & Comments, Ratings & Reviews, Share Plugin
Bunchball [Bun]	Competition, Economy/Marketplace, Feedback, Levels & Missions, Profiles, Progress Bar, Ranks & Badges, Teams, Time Pressure, Activity Feed, Community Forums, Share Plugin, Social Login
beintoo [bei]	Competition, Economy/Marketplace, Activity Feed, Community Forums, Likes & Comments, Ratings & Reviews, Share Plugin, Social Login
UserInfuser [clo]	Leaderboard, Widgets, Activity stream, Badges, Points, User profile
IActionable [iac]	Competition, Progress Bar, Ranks & Badges, Time Pressure, Ratings & Reviews
CrowdTwist [cro]	social loyalty, rewards
Manumatix [man]	Competition, Activity Feed, Social Login
Gigya [gig]	Economy/Marketplace, Levels & Missions, Profiles, Ranks & Badges, Activity Feed,Likes & Comments, Live Chat, Ratings & Reviews, Share Plugin, Social Login
Punchtab [pun]	Economy/Marketplace, Feedback, Levels & Missions, Profiles, Teams, Activity Feed, Community Forums, Share Plugin, Social Login
Keas [kea]	Teams, Likes & Comments, Ratings & Reviews, Social Login
RepTivity [rep]	Competition, Feedback, Levels & Missions, Activity Feed, Community Forums
RoarEngine [roa]	Competition, Economy/Marketplace, Feedback, Levels & Missions, Profiles, Progress Bar, ctivity Feed, Community Forums,Ratings & Reviews, Social Login
Gaminside [gam]	Feedback, Levels & Missions, Progress Bar, Ranks & Badges
Kudos Badges [kud]	Competition, Economy/Marketplace, Ranks & Badges, Teams, Likes & Comments, Ratings & Reviews, Social Login
Mplifyr [Mpl]	Competition, Feedback, Levels & Missions, Ranks & Badges, Teams, Activity Feed, Likes & Comments, Ratings & Reviews, Social Login

namic scenario, a service based approach is preferred. In a service based approach, a set of game design elements are made available as RESTful web services. Any developer while gamifying an application can use these services with minimal development effort required.

Earlier studies suggest that gamification effectively improves adherence to coding conventions of the code written by developers [PJ15]. Table 2 in the Appendix section lists gamified applications in Healthcare, E-commerce, Manufacturing, Education and SE domains and game design elements used within. A comprehensive list of domains, gamified applications, frameworks supporting gamification and game design elements are presented in tables 1, 2 and 3.

Existing frameworks that support gamification tend to be application specific or game design element(s) specific. Open source frameworks like Mozilla OpenBadges and UserInfuser provide programmable interface. These programmable interfaces require developer to understand and tweak the code base of the frameworks.

2.3 Game Design Elements

Game design elements are tools and techniques required to gamify an application [1]. They play an important role in determining the success of a gamified application in any domain. Gamification design process is the process of designing and applying game design elements to create the rules and modes of operation for a gamified application. For supporting gamification of software engineering activities, any platform supporting the gamification of applications should support invocation and usage of any/all the game design elements required to gamify those activities. Pedreira et al. [10] found out that *"points" "Badges"* are the most commonly used game design elements.

After a comprehensive study, we collated a generic set of game design elements applicable over various domains (shown in Table 4). Table 4 is derived from tables 1, 2 and 3. Following the SLR template specified in Appendix, by eliminating the duplicates in each of the Tables 1, 2 and 3, we were able to derive Table 4.

All the game design elements are expected to act as stimulus in promoting interest among users. However, we identified one game design element i.e. (*social network connection between users*) that can assist in retaining users if its not supported within a gamified application. For example, let's consider the instance of Quora and StackOverflow. StackOverflow, intended to be a purely technical Q&A site achieved the best quality without allowing its users to be able to follow similar users or users whom a person is in-

Table 2: Classification of Applications available and Domains

Gamified application	Domain
CodeBrag, collabreview, GitPoints, Getbadges	Software Engineering, collaborative learning
Treehouse, code hunt, code combat	Goal Tracking and Proof of Achievement
Mint.com [minb]	Fostering Financial Independence and Goal Tracking
Recyclebank [rec]	Customer Loyalty and Rewards
Step2 [ste]	Customer Loyalty
Keas [kea]	Employee Wellness, Cost Reduction
The World Bank [urg] – Evoke	Solving World Problems
DevHub	Project Completion
4food [4fo]	Social Loyalty, Brand Awareness, Engagement, and Social Missions
Proof [mina]	Motivation, Goal Tracking
Engine Yard [eng]	Customer Engagement
Beat the GMAT [bea]	User Engagement, Goal Tracking and Competition
Bluewolf [blu]	Employee Engagement and Motivation
ChoreWars [cho]	Healthy Competition, Employee Motivation
SCVNGR [scv]	Building Brand Awareness, Team Building
The Email Game [ema]	Brand Awareness, Team Building, Productivity

terested in. While Quora also being a Q&A site, but with different goal of getting any question answered with real life experiences could afford to let users follow people. Many times the most accurate answer on Quora is not on the top list of answers for a question, whereas Stackoverflow has the most accurate answer which is verified to be a working solution on top always. This pattern can be related with answers that attain "upvotes" based on social influence rather than real value add to the question. Additionally, the gamified application should remain fair to all the users without being manipulated or misused for the personal benefit of one particular user of the game.

Another specific game design element that can be designed as a service is "play-privately". To implement this as a service, three different views of users should be implemented namely, *Admin*, *private user* and *public user*. Hence the service is not atomic to itself and can be implemented as a composite of "user profile", "rewards", "badges", "leaderboard" and other dependent services. A user is given a choice to make his profile public or private. In case of private, the user shall still be able to see the public profiles and compare his ranking with the public profiles. The "public profile" user will not be able to see the details of a private user. However "admin view" will have details of all the users and the data will be used while the global leader board display is implemented.

Table 3: Classification of SE activity and gamified Applications

Activity	Gamified application
software requirements	collabreview [PJ15]
configuration management	Devhub [dev]
Implementation	code combat [coda], Treehouse [tea], code hunt [codb]
knowledge management	
Code Inspection	codebrag, gitpoints, getbadges
Documentation	collabreview

The other important design element (although not a game design element per-se) , 'FAQ'(Frequently asked questions) section should be included in any game. This should also include the explanation about the logic behind awarding game design elements to the user (player). This helps user understand how to progress through levels during the game and allows user to sail in the intended direction of gamifying an application.

2.4 Service orientation

Service orientation facilitates the architecture to be implemented as a set of loosely coupled components. However, in a service oriented architecture the consumer will continuously poll requests to the provider of the services. This can result in unnecessary wastage of computational resources. Events and Hooks can overcome this problem by informing the consumer when to poll a request. Earlier studies [8] [18] [6] suggest this approach of communication between producer and consumer reduces resource utilization. We inherit features from this architectural style while designing the extensible architectural framework for gamifying SE activities.

3. ARCHITECTURAL DESIGN AND EVALUATION

3.1 Gamification, Product line engineernig and SCV analysis

A major objective of our work is to facilitate service orientation to maximize the re-use of components while implementing the generic architectural framework. We utilize ideas from Product line engineering to achieve our goals. Using a product line approach [16], [9] helps build re-usable components while building a series of applications. The approach consists of three phases: (1) Product Management

Table 5: Classification of Game design elements.

Classification			
	Behavioural	Feedback	Progression
Static	Achievements, Rewards	Checklists, Milestones	Points, Badges, Medals, Levels
Dynamic	Leaderboard, Teams	Activity stream	Missions, Countdown, Recommendations, Progress bar

Table 4: Game Design Elements and Domains

Domain	Game Design Elements	Applications
Branding and customer engagement	Avatars, Points, Leaderboards, Achievements, Badges, Levels, Missions	Samsung Nation, YoungOffice, Insider
Customer Loyalty, Education and Marketing	Social Network, Rewards, Avatars, Points, Leaderboards, Achievements, Badges, Levels	FourSquare, Teachcode
Employee wellness, Causes, Environment Recycling	Real prizes, Virtual Currency, Facebook credits, Virtual goods, Gifting, Sharing, countdown, Teams	Hallmark, Megagame
Others	Contests, Tactics, Recommendations, Activity Stream, Notifications, Followers, Play privately, Bonus, Medals, Game Analytics, Milestones, Accomplishments	Others

Phase (2) Domain Engineering Phase and (3) Application Engineering phase.

3.1.1 Product Management

We define the "scope" i.e a product family of gamified applications for software engineering. The limits of our architectural framework are defined to include support of game design elements that are suitable for but not restricted to software engineering domain. The framework can later be extended to support game design elements required for other domains. However, it needs to be noted that if the domain is large the establishing across different applications within a domain becomes difficult.

3.1.2 Domain Engineering phase

James Coplien et al. [3] proposed an approach called Scope, Commonality and Variability (SCV) analysis to identify the common and variable aspects in a given scope, while working with a product line of systems. We analyzed a list of gamified applications in software engineering domain as well as other domains (shown in tables previously). We studied the work by Pedreira et al. [10] which lists down the systematic mapping between software engineering and gamified applications. Table 4 lists down the game design elements identified among the existing applications and prior literature in the field of gamification.

SCV analysis is used to identify the re-usable components that are common across all realizations (Gamified applications) developed using the architectural framework. After listing down the common and variable game design elements across domains, Game design elements are fine tuned to those that are specifically applicable to software engineering activities represented in the feature Matrix (Table 5). However, it needs to be noted that additional game design elements may be added as needed. These game design elements are further classified into static and dynamic categories based on the implementation of these as services. The services are served by the *provider* depending on the user input given by *consumer*. Static game design elements when called will return the same value irrespective of the state of the machine. Dynamic game design elements are the ones whose output will depend on the earlier execution state of the same or different function within the application/*consumer*. Static game design elements such as a unique UI common to the product family is implemented. A leader board and activity stream along with user profile are common game design elements. These are implemented as part of generic architecture to improve the re-use of code base.

Both static and dynamic game design elements can be broadly classified into three categories namely: Behavioural, Feedback and Progression. *Behavioural* game design elements are usually focused on human behaviour and interaction with system. Depending on the player action and simulation *Feedback* design elements will be awarded to the user to improve the motivation of the user. *Progression* design elements define the state and structure of the gamified application.

3.1.3 Application Engineering

A realization of the architecture may or may not require all the game design elements. In other words, specific game design elements like points, badges, play-privately may be needed for realizing some software engineering activities (for example, code review and bug reporting) while some others might be considered for other realizations. Also, we noticed that there is no single set of mandatory game design elements that are suitable for all the software engineering activities. Hence, gamification of any software engineering activity will be a composition of a subset of services listed in table 4.

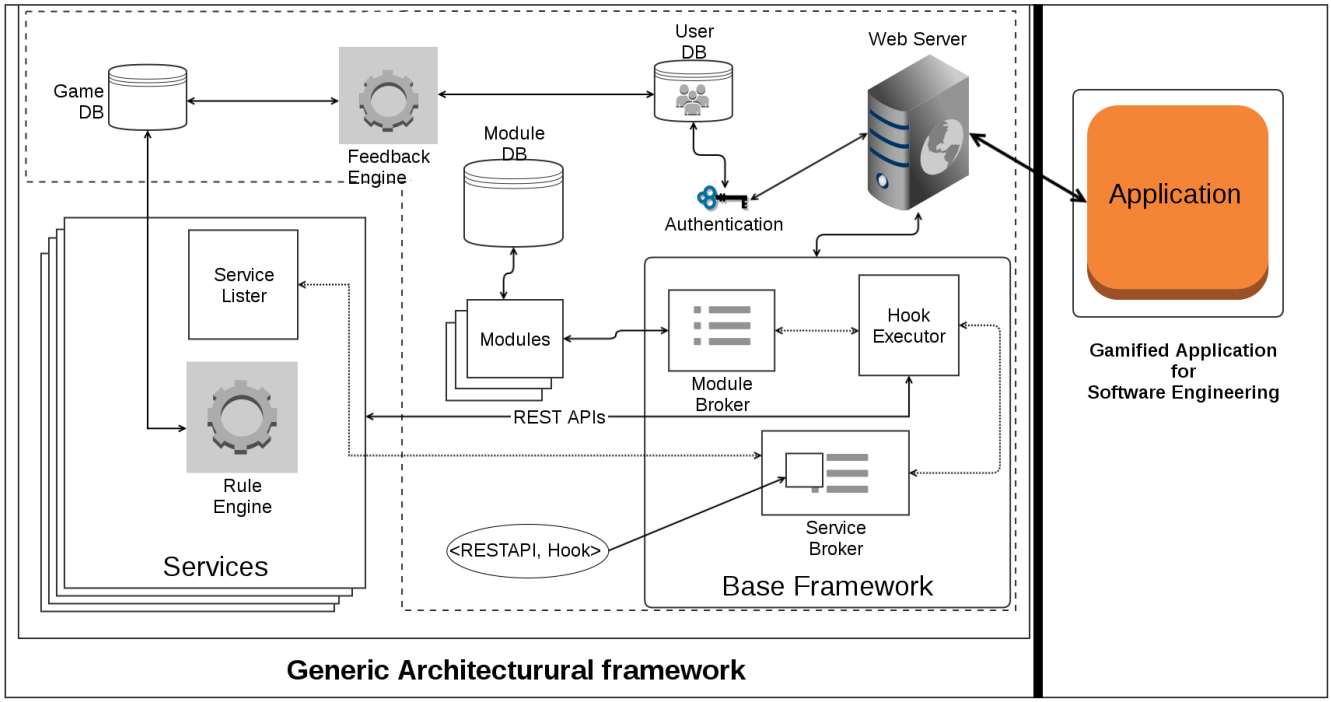

Figure 1: Architecture Diagram.

3.2 Architectural Framework

The architectural framework facilitates separation of static and interactive (dynamic) game design elements by implementing them as RESTful web services. The service based architecture shown in figure 1 consists of two layers. First layer consists of **Base Framework** along with **database utilities** and common services that are required to generate a gamified application which does not perform any SE activity. In order to generate a custom gamified application suitable for gamifying SE activity, the functionality provided by **services layer** should be incorporated. This will lead to realization of desired gamified application. The *Services* layer provides all the game design elements which are required to gamify software engineering activities, but all of these services may not be required for any single instance of gamification.

Static services are mostly common across realizations and hence the implementation of the framework can directly be inherited. Unlike static services, dynamic game design elements are implemented as services which facilitate the option to tweak and extend the implementation by developer of gamified application.

Figure 1 shows our extensible architecture for gamifying software engineering activities. The extensibility is made possible by the highly decoupled modules facilitating incorporation of other available modules as needed. The key parts of the architecture are : Services, Modules, Service Broker, Module Broker, Hook Executor, Rule Engine, Feedback Engine, Databases and Web Server.

Authentication module: The User sending request to application should be authenticated before performing any interaction with application. Support for authentication module is provided using openid [7] or Mozilla's persona [8].

Base framework: It consists of all the common game design elements identified during the domain engineering phase3.1.2 by performing commonality analysis over various domains shown in table 2 . Along with the common game design elements, the architecture provides support for user interface elements such as charts, tables, forms using ember.js [9] a javascript based mvc framework. These elements can be dynamically consumed by modules and services, thus saving developer's effort.

Modules: Various utilities required to perform software engineering activities such as code review, documentation, bug tracking, etc exist as separate modules in our architecture. Modules can be disabled and enabled as and when required. Whenever a new module is introduced, the module will register its supported URLs, hooks and functionality with "Module Broker".

Module Broker: Module broker is an API for loading and interacting with modules.

Services and Service Lister: The game design elements are modeled as services. Services are responsible for providing the game design elements as REST APIs. The *Service Lister* communicates on behalf of service with the base architecture to register URL namespace, and it also provides sample <REST API, Hook> map to Service broker. Sample REST APIs provided by services are shown in Table 6. This table includes API for "badges" and "leader board" game design elements.

[7]http://www.openid.net/
[8]https://www.mozilla.org/en-US/persona/
[9]http:///www.emberjs.com

Table 6: Game design element (Service) end-points

HTTP Method	URI	Action
GET	http://[hostname]/api/badges/badge/[badge_id]	Retrieve image of a badge
GET	http://[hostname]/api/badges/user/[user_id]	Retrieve list of badges award to a user
POST	http://[hostname]/api/badges/event	Share event with badges service
GET	http://[hostname]/api/leaderboard/get	Retrieve stats
POST	http://[hostname]/api/leaderboard/event	Share event with leader board service

Service Broker: Service broker is responsible to communicate with services on behalf of base application. The Service Broker also contains <REST API, Hook> map from service listers, which is updated automatically as and when a new service is added to the generic framework. Except this, the base framework provides application developer the freedom to update <REST API, Hook> mapping according to the need of the application.

Hook Executor: Each of the user actions is broadcast as an event. Hook executor holds the responsibility in our architecture to successfully execute all assigned tasks on each event. Along with this, it calls all services listening to that particular event by invoking lookup on <REST API, Hook> map. Asynchronous requests made to services are handled by Hook Executor.

Rule Engine: A rule engine is required to reward the user according to the rules and regulations defined for a gameplay. Depending on the user activity or event the rule engine alters the game design elements associated with a user's profile. The rules for each gamified application varies and are different among each realization of the architecture, as defined by the application developer.

Feedback Engine: Any gamified application, must consist of feedback mechanism. A suitable way of providing a Feedback Engine is through an event-driven approach. In our architecture, Feedback Engine will be a "server" that runs continuously in the background, collecting data from the rule engine and send information that will persuade users either via communication channels like mails or using *Activity stream* game design elements.

Databases: We configure three separate database schema to support Gamified data, Module data and User data. The data will be visible to user/admin depending on the user preference. Also, to implement "play-privately" as a service, the Game database scheme needs a minor modification, i.e. a new column "public", with value either 0 or 1. thus, creating a logical layer above normal Database schema.

A composition of services (game design elements and game qualities) when applied on applications performing software engineering activities can turn them into gamified activities. As an example instance, we have instantiated the generic architecture for gamifying the tools that support code review software engineering activity.

Hooks are special kind of functions, which when called broadcast events. As described above, all modules during initialization register hooks implemented in them with Module Broker, by passing <Function Name, Hook Name> map. Now, collection of these maps is used to decide which functions are listening to a particular broadcasted event, and are executed accordingly. We have followed this as a core concept for implementing different modules. It minimizes the developer's time by providing relevant end-points quickly (in the form of events) to execute newly written function. This process can be best described as Event driven Service-oriented architecture. [18] [6]

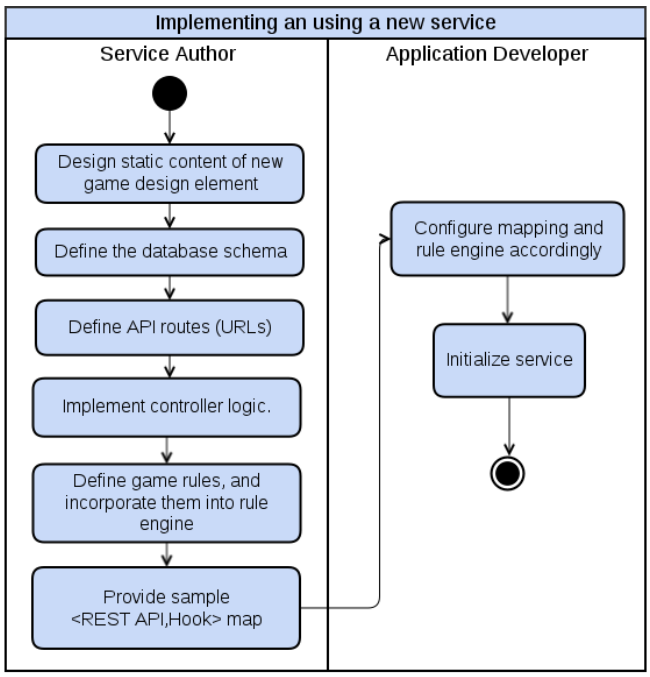

Figure 2: Implementing a new service.

3.3 How to implement and use a new service?

The game design elements are designed and served as RESTful web-services. This in addition to the usage of MVC based framework enables extensibility in the architecture. All the services have URL namespace assigned to them. A URL Namespace is used for allocating unique URL prefix to every new service, saving any possible routing conflict. We have implemented five game design elements to validate our approach. We implemented *Badges*, Leaderboard, User Profile, Activity Stream and Timeline as services. New game design elements can be designed and implemented as shown in Figure 2. The implementation process can be repeated for any new game design element. For example, in case of Badges, the image of specific badge can be designed using a good image editing tool. Once the static content is made

available, host the content on server from where nginx[10] can serve this content. For this service to be consumed by a gamified application and to assign the value to user profile, the metadata of the game design element should be maintained in a database table. This data will be copied to user table every time the game design element is awarded to a user profile. This requires design of database schema for the game design elements and storage of the metadata in game database. To serve the game design element as a RESTAPI, the route should be uniquely defined. The Controller Logic used to invoke the game design element shall be implemented within the service itself. Sample <REST API, Hook> mappings are provided to application developers. The developer should define constraints to call the Hooks, by tweaking these sample mappings. After these steps, the architecture would successfully support the newly implemented game design element.

The extensible architectural framework has been implemented by the authors. The framework is being extended as and when a new game design element is required for realizing a new gamified instance of an SE activity. Such an extensibility facilitates crowd sourcing the development of a new service by anyone apart from the authors of the framework. This also helps us to iteratively develop a new service (game design element) as and when required. The initial framework is currently made available at github link (https://github.com/anhsirksai/ayuta).

3.4 Gamification of code review process

Based on the architecture proposed, we realized the architecture and instantiated the modules required for a gamified code review application. Game design elements like Ranking system, Levels and Missions, Progress bar, Counting the number of likes for activity and favourite comments, Bonus, Badges, points and rewards can all be used as incentive constructs. In our gamified application we started with implementing Badges as key incentive construct for code review application.

The realized architecture for code reviews consisted of the following components:

- Web server : Nginx
- Database : Mysql
- Frameworks : Django, Ember.js, Flask
- Modules : diff utility
- UI requirements : Tables
- Services : Badges, Leader Board, User Profile, Timeline, Activity stream.

For realization of the architecture, we started with implementation of base framework consisting of authentication module, Module broker, Hook executor and Service Broker. Flask is a python micro framework and enables implementation of restful web services in python even by a novice python developer. As there are no rigid scaffolding rules and requirements, Flask provides more flexibility. For this very same reason we chose Ember.js the javascript mvc framework over the popular anjular.js. We chose Nginx over apache as web server, as Nginx is the best asynchronous server, is event-driven and handles requests in a single (or possibly very few) threads. Nginx provides Static file serving, Reverse proxying, Load balancing, Server-side includes FastCGI. Nginx is able to serve more requests per second with less resources because of its architecture. All these are a necessity for the dynamic applications (such as gamified SE activities) to be implemented. Although *Apache HTTP Server* is considered to be more suitable for dynamic websites, our requirement of handling multiple requests to

[10]http://nginx.org/en/

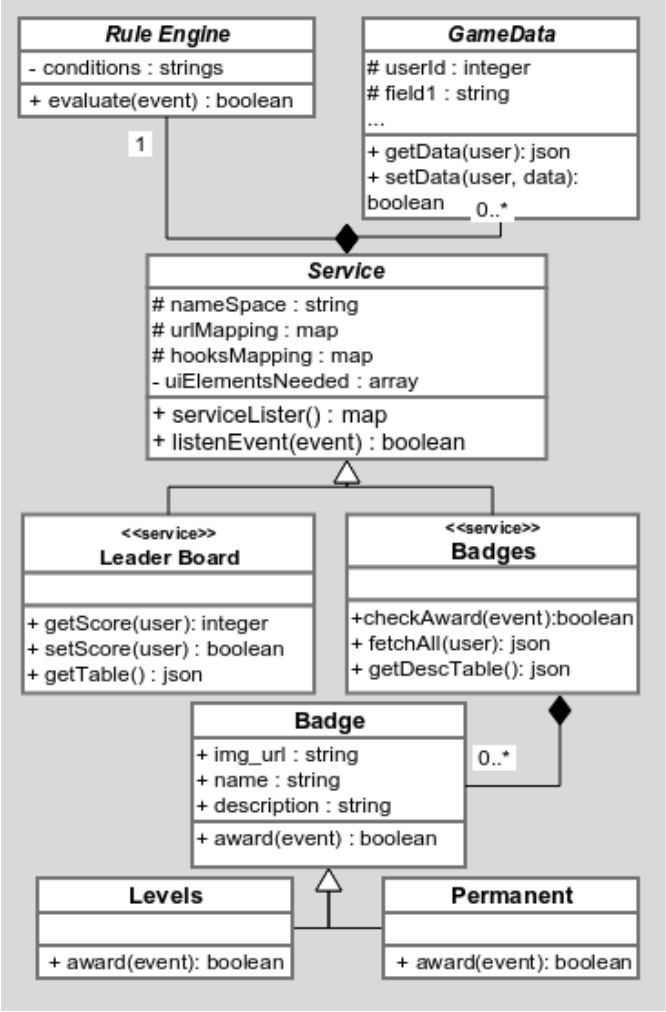

Figure 4: Class diagram showing usage of Factory design pattern while realizing a runtime instance using the generic architectural framework.

serve various game design elements in parallel pushed us towards Nginx. The UI elements were kept as different Ember components [11]. The base framework was then extended by adding *module database. "diff utility"* module (extracted from open sourced diffcommenter[12]) and code review module were implemented. The code review module provides hooks namely `cr-comment-posted`, `cr-comment-edited`, `cr-comment-deleted`, etc. that can be used for gamification purposes.

For the code review instance, we have identified a set of static and dynamic game design elements that are to be provided as RESTful web services. The list of game feature services are: in the list of static category - *points, checklists, rewards, badges, medals, levels, milestones and Upvotes* and in the list of dynamic category (Leaderboard, Progress bar, Activity stream, Recommendations and Realtime Feedback). The initial scope included only badges and leader board. While realizing badges and leader board as services, we identified large chunks of duplicate code within the implementation of these two services. To reduce the

[11]http://emberjs.com/api/classes/Ember.Component.html
[12]https://github.com/Babazka/diffcommenter

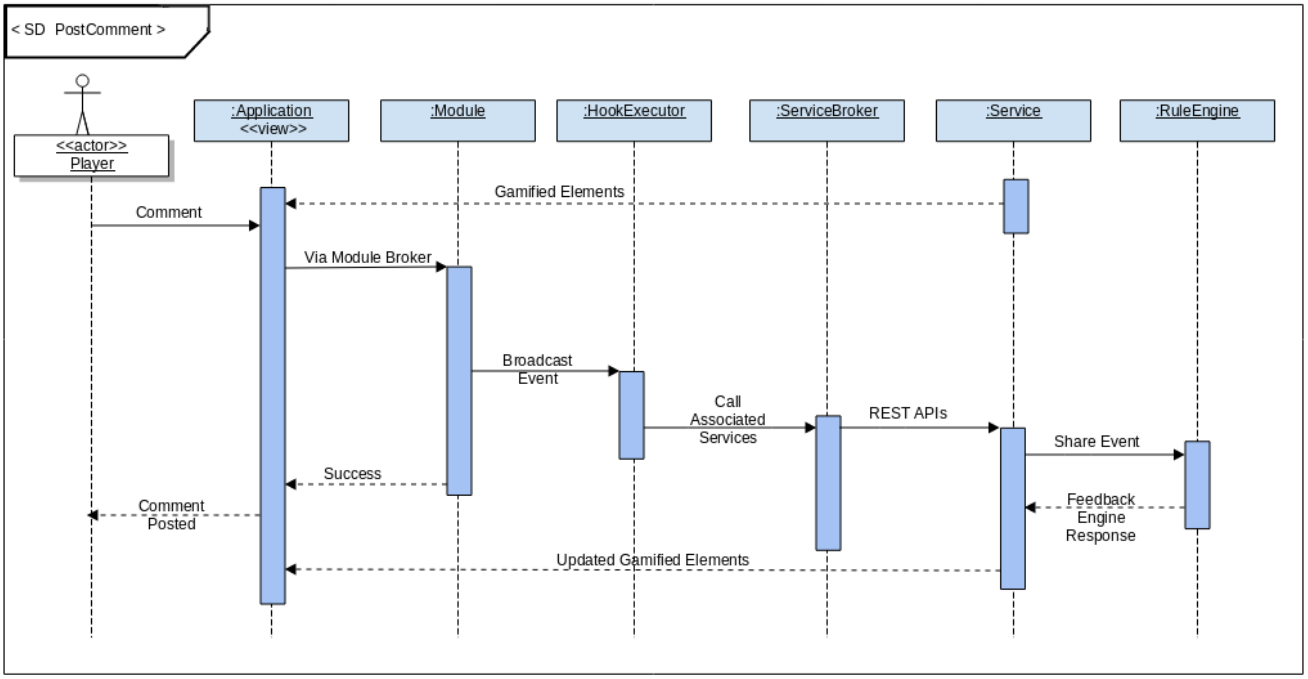

Figure 3: Sequence diagram showing the control flow to award a game design element to user profile.

duplication effort we incorporated "Factory Design Pattern" [5].

Eventhough an earlier study [4] suggested to avoid factory design pattern, the usage of factory pattern within base service class Figure 4 benefited us by reducing the development efforts during early stages of implementation. It also provided additional benefits of implementing additional game design elements like *avatar* game design elements along with initial scope of 5 game design elements. The services interact with the service broker over REST APIs for registering <RESTAPI, Hook> map when the service is enabled, and then Hook executor calls the APIs as and when required. The working of Hook executor, can be seen in Figure 3 for one such activity user performs i.e. post new comment on the gamified software engineering tool.

The instantiation and implementation of such an architecture has its own set of challenges. In general, the challenges identified while building a gamification engine are divided into: Integration problems (like: Front end integration, Rules and business logic Integration) and Feedback process [2]. In our architecture, we have a separate rule Engine and feedback Engine that addresses the challenges mentioned.

3.5 Evaluation of Generic Architectural Framework

To validate the extensibility of our proposed architecture, we implemented multiple services and a run-time instance for code review process that made use of some of these services. However, a thorough validation requires a comparative study on the usability of the tool generated against existing tools.

As described earlier in the Figure 2, implementation of a new service does not require implementing from scratch, instead we can re-use the existing code base implemented following "factory pattern". The extensibility of our architecture leads to support of utilities (like "difftool") required for SE activities. If for any SE process, underlying utility support is missing, the new module3.2 can be added within the generic architecture, since we run our architectural framework on linux server. This approach of implementing game design elements gives us the flexibility to use the services as plug-and-play modules while implementing a new gamified software engineering application. Extensibility, in terms of developing gamified versions of other SE activities using our framework is on-going.

We have realised a code review game using the generic architectural framework. The gamified application currently supports review of code, creation and management of user profiles, activity stream of all the registered users is displayed. Apart from these, a leader board is initialized from the generic framework and "badges" as a service is being utilised.

The framework can be evaluated from the perspective of the authors, where the proposed architecture should function as proposed. From the perspective of developer implementation time and efforts should noticeably come down to be able to justify the notion of extensibility. From the perspective of end-user, the gamified application should be successful in retaining the players interest and persuading him to come back to perform the same task again and again.

Other methods such as Architecture Trade-off Analysis Method (ATAM), System Architecture Tradeoff Analysis Method (SAAM), System of Systems (SoS) Architecture Evaluation, Active Reviews for Intermediate Design that measure the quality of architecture were studied. But our focus is on evaluating the extensibility of the architecture. Hence these methods were not relevant.

4. THREATS TO VALIDITY

In our work, we concentrate on game design elements suitable for but not restricted to Software engineering domain. The services we support can also be used to gamify applications in other domains as well, but at this juncture our framework has not been used/tested to sufficiently support all the required game design elements in other domains.

Our approach also deals with integrating all the available services already implemented and existing (like : gravatar). We also extend the services that are available so as to suit our requirements. The Webpage or the UI for our gamified application is not developed from scratch, instead we use a bootstrap template. Hence for our application to load completely, a working internet connection is required. The possibility of building a stand-alone gamified application is not supported.

The extensibility of our approach is validated in building a new service. Description about building a new service is provided in section 3.3. Although our architecture currently supports "diff utility" and "wikipages", rest of the *utilities* required to support every SE activity should be incorporated into the architecture. We are still to build multiple runtime instances (Gamified applications for SE processes other than code reviews) using our framework. Our current work deals with only one instance i.e *Code reviews*, In future, we intend to implement other run-time instances using our framework and also evaluate the usability of our framework.

The usability of the tool generated using the architectural framework is currently being studied. We are conducting a study to compare the existing gamified code review tools to the instances we generated. We plan to publish these results as future work.

Metrics to measure the extensibility are being studied. ISO9126 has metrics for maintainability and functionality. The main properties ISO9126 check for are analyzablity, stability, changeability, testability. Changeability of game design elements such as personal scores, group incentives, leader board implemented within a gamified application can be studied. We plan to take up re-usability metrics to evaluate the architectural framework.

In our work, to realise the architecture, we started implementing services that are most commonly used game design elements, i.e., badges. We continued to implement other required services to gamify a software engineering process task. Hence our focus is to first implement the services that are required to gamify the *code review process*. We plan to implement the remaining services stated in Table 5 to support the gamification of software engineering tasks. However this requires substantial effort. So, crowd sourcing the development of services is a possible option.

5. LIMITATIONS AND FUTURE WORK

While performing sc&v analysis, we have identified game design elements used in various domains. Our work is based on defined set of domains, which may not be representative of all the domains. Further work can be done on expanding the list of game design elements with in a domain and across multiple domains to make it more exhaustive. Writing each of these game design elements as a service is another resource intensive task. At present there are few game design elements that are offered as services. Implementing all the services is best achieved via crowd sourcing.

Designing a layered event driven service based architecture is challenging, especially when the game design elements and game qualities are implemented as non-orthogonal services. A complete set of utilities that are required to support software engineering activities are to be identified and incorporated into the platform. This can be taken up as background study and we plan to improve our architecture accordingly in future. . In our study, we have assumed that the services are truly non-orthogonal, hence order of composition of services may not matter. However, this may not be true. Hence a study needs to be done on the compositionality of services.

In future, we plan to realize more architectural instances to gamify additional software engineering activities. We also plan to gamify another instance of software engineering activity like a process model (agile/iterative/spiral) suitable for academic and start-up projects. Designing a complete gamified platform for a software engineering activity involves gamifying legacy applications that support software engineering activities or coming up with a new tool that is gamified from scratch. Once game design elements are available as services, we intend to host these as RESTful web services and encourage software engineering community all across the world to take advantage of our frameowrk and gamify various activities.

We are currently working on getting sophomores (second year undergraduate students) to use the gamified application developed and get the feedback regarding the usability of the tool and advantages of using a gamified tool. We plan to conduct a study on developing gamified instances using our architecture and additionally measure the productivity of developers who use the gamified instances.

Our Architecture facilitates a decoupled approach wherein the actual UI of an application supporting the software engineering activity and the non gamified UI remains the same. We allow application developers to come up with another layer of UI for the gamified version. This at times can be perceived as a limitation, since the developers have to work with two different user interfaces. The design of gamified UI supported across various devices of different sizes involves designing of UI in compliance with web 2.0 standard. As of now, we have not included any such modules where a user will be guided and restricted to develop an interface in compliance with HTML5 (web2.0) standards.

6. REFERENCES

[1] E. Adams. *Fundamentals of Game Design*. New Riders Publishing, Thousand Oaks, CA, USA, 2nd edition, 2009.

[2] M. Ameling, P. Herzig, and A. Schill. A generic platform for enterprise gamification. In *Software Architecture (WICSA) and European Conference on Software Architecture (ECSA), 2012 Joint Working IEEE/IFIP Conference on*, pages 219–223, Aug 2012.

[3] J. Coplien, D. Hoffman, and D. Weiss. Commonality and variability in software engineering. *Software, IEEE*, 15(6):37–45, Nov 1998.

[4] B. Ellis, J. Stylos, and B. Myers. The factory pattern in api design: A usability evaluation. In *Software Engineering, 2007. ICSE 2007. 29th International Conference on*, pages 302–312, May 2007.

[5] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2002.

[6] S. Y. Ghalsasi. Critical success factors for event driven service oriented architecture. In *Proceedings of the 2Nd International Conference on Interaction Sciences: Information Technology, Culture and Human*, ICIS '09, pages 1441–1446, New York, NY, USA, 2009. ACM.

[7] P. Herzig, M. Ameling, and A. Schill. A generic platform for enterprise gamification. pages 219–223, 2012.

[8] Z. Laliwala and S. Chaudhary. Event-driven service-oriented architecture. In *Service Systems and Service Management, 2008 International Conference on*, pages 1–6, June 2008.

[9] L. Northrop. Introduction to software product lines. In J. Bosch and J. Lee, editors, *Software Product Lines: Going Beyond*, volume 6287 of *Lecture Notes in Computer Science*, pages 521–522. Springer Berlin Heidelberg, 2010.

[10] O. Pedreira, F. Garcia, N. Brisaboa, and M. Piattini. Gamification in software engineering, a systematic mapping. *Information and Software Technology*, 57:157–168, January 2015.

[11] C. R. Prause and M. Jarke. Gamification for enforcing coding conventions. ESEC/FSE 2015, pages 649–660. ACM, 2015.

[12] D. Rojas, B. Kapralos, and A. Dubrowski. Gamification for internet based learning in health professions education. pages 281–282, July 2014.
[13] N. Tillmann, J. De Halleux, T. Xie, S. Gulwani, and J. Bishop. Teaching and learning programming and software engineering via interactive gaming. In *Software Engineering (ICSE), 2013 35th International Conference on*, pages 1117–1126, May 2013.
[14] A. Uskov and B. Sekar. Serious games, gamification and game engines to support framework activities in engineering: Case studies, analysis, classifications and outcomes. In *Electro/Information Technology (EIT), 2014 IEEE International Conference on*, pages 618–623, June 2014.
[15] B. Vasilescu, A. Capiluppi, and A. Serebrenik. Gender, representation and online participation: A quantitative study. *Interacting with Computers*, 26(5):488–511, 2014.
[16] D. M. Weiss and C. T. R. Lai. *Software Product-line Engineering: A Family-based Software Development Process*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1999.
[17] I. Whitepaper. Serious solutions with serious games straining with complex systems. 2011.
[18] Q. Zagarese, G. Canfora, E. Zimeo, I. Alshabani, L. Pellegrino, and F. Baude. Efficient data-intensive event-driven interaction in soa. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing*, SAC '13, pages 1907–1912, New York, NY, USA, 2013. ACM.
[19] G. Zichermann and C. Cunningham. *Gamification by Design: Implementing Game Mechanics in Web and Mobile Apps*. O'Reilly Media, Inc., 1st edition, 2011.

APPENDIX

A. REFERENCES

[4fo] 4food. http://4food.com/.
[bad] badgeville. http://badgeville.com/.
[bea] beatthegmat. http://www.beatthegmat.com/.
[bei] beintoo. http://www.beintoo.com/.
[big] bigdoor. http://bigdoor.com.
[blu] bluewolf. http://www.bluewolf.com/.
[Bun] Bunchball. http://bunchball.com/.
[cho] chorewars. http://www.chorewars.com/.
[clo] cloudcaptive. http://www.cloudcaptive.com/.
[coda] codecombat. https://codecombat.com/.
[codb] codehunt. https://www.codehunt.com/.
[cro] crowdtwist. http://www.crowdtwist.com/.
[dev] devhub. http://www.devhub.com.
[ema] emailga. http://emailga.me/.
[eng] engineyard. https://www.engineyard.com/.
[gam] gaminside. http://www.gaminside.com/.
[gig] gigya. http://www.gigya.com/.
[iac] iactionable. http://iactionable.com/.
[kea] keas. http://www.keas.com/.
[kud] kudosbadges. http://www.kudosbadges.com/.
[man] manumatix. http://manumatix.com/.
[mina] mindbloom. http://www.mindbloom.com/proof.
[minb] mint. https://www.mint.com/.
[Mpl] Mplifyr. http://www.mplifyr.com/.
[PJ15] Christian R. Prause and Matthias Jarke. Gamification for enforcing coding conventions. ESEC/FSE 2015, pages 649–660. ACM, 2015.
[pun] punchtab. http://www.punchtab.com/.
[rec] recyclebank. https://www.recyclebank.com/.
[rep] reptivity. http://www.reptivity.com/.
[roa] roarengine. http://roarengine.com/.
[scv] scvngr. http://www.scvngr.com/.
[ste] step2. http://www.step2.com/loyalty/.
[tea] teamtreehouse. https://teamtreehouse.com/.
[urg] urgentevoke. http://www.urgentevoke.com/.

B. SLR TEMPLATE

Table 7: SLR Template

1.Frame research questions.	
2. Document search	1. Search string 2. Search sources : INSPEC | El Compendex | Science Direct | Web of Science | IEEExplore | ACM Digital library 3. Publication Bias and how to overcome ? The editorial predilection for publishing particular findings e.g., positive results, which leads to the failure of authors to submit negative findings for publication . Researchers formulate only true hypotheses
3. Selection Criteria	- Inclusion and exclusion criteria - Quality assesment of studies.
4. Reporting	Dissemination can be done in two ways: • Publish at conferences/journals like – Journals ∗ Information and Software Technology ∗ IEEE Transactions on Software Engineering ∗ Empirical Software ∗ IEEE Software – Voice of Evidence – Conferences ∗ ESEM ∗ EASE • Technical Report or section of a PhD/MS thesis
5. Report Structure	• Structured abstract • Background • Research questions • Method • Included and excluded studies • Results • Discussion • Conclusions
6. Possible graphical representations	• Forest plot • Bubble plot • L'Abbé plot • Galbraith (radial) plots (or) • Just a table